PROJECT SYNOPSIS

ProjectHub

Collaborative Project Development, Review & Showcase Platform

MERN Stack | Role-Based | Student - Supervisor - Admin

Group Leader	Prince (Roll No. 2401301022, Section B)
Group Members	Nishant (Roll No. 2401301149, Section B) Pushp (Roll No. 2401301143, Section B)
Course	B.Tech Computer Science & Engineering
University	Geeta University, Panipat
Subject	Project-I (Semester Project)
Technology	MongoDB, Express.js, React.js, Node.js

1. Introduction

ProjectHub is a full-stack web platform where students build and manage their academic and personal software projects, supervisors review them and give structured feedback, and administrators manage the entire platform. It combines a project showcase, lightweight project management, a supervisor review system and a developer community in a single application.

The system is built on the MERN stack (MongoDB, Express.js, React.js, Node.js) and uses JWT authentication with role-based access control. The scope is deliberately controlled: ProjectHub is not intended to replace GitHub or Jira, but to give students a focused place to document, track, submit and showcase their work.

2. Problem Statement and Need

In most colleges, project work is tracked through scattered tools such as WhatsApp groups, spreadsheets, emails and paper reports. This leads to the following problems:

- No single place for students to manage project details, team members, tasks and progress.
- Supervisors cannot easily see the progress of every student or give consistent, recorded feedback.
- Review scores and feedback are not structured, so evaluation is subjective and hard to compare.
- Completed projects are not preserved or showcased, so good work is lost after submission.
- Administrators have no overall view of users, projects, activity and platform statistics.

ProjectHub solves these problems with one organised, role-based platform covering the complete life cycle of a project, from idea to approval and showcase.

3. Objectives

- Build a polished full-stack MERN application that demonstrates real-world development skills.
- Provide secure registration and login with JWT and bcrypt, and redirect each user by role.
- Let students create projects, add technologies and team members, manage tasks and track progress.
- Give supervisors a dashboard to track students, review projects, score them and approve or request changes.
- Give administrators tools to manage users, moderate projects, view reports and analytics.
- Rank projects and developers fairly using a backend-calculated score that users cannot manipulate.
- Notify users of important events such as submissions, reviews and approvals.

4. Scope of the Project

In Scope	Out of Scope (Future Scope)
Authentication and role-based authorization	Real-time chat
Student profile, project CRUD, team and tasks	Payment system
Supervisor review, rating and feedback	Microservices, Redis, Docker orchestration
Admin user and project management, reports	Advanced AI features, video conferencing
Explore, search, filters, likes, comments, bookmarks	Complex GitHub automation, recommendation engine
Leaderboards, notifications, analytics dashboards	Mobile application

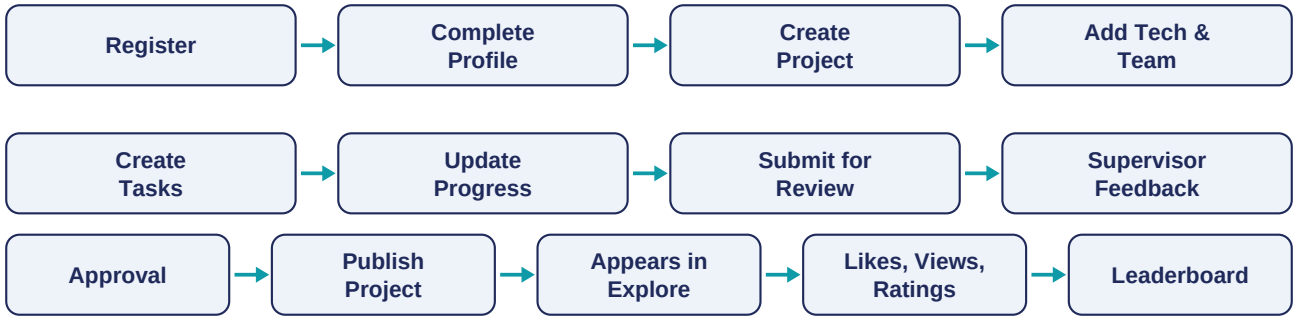
5. Proposed System and User Roles

The platform has three user roles. Each role has its own dashboard and permissions, enforced on both the frontend routes and the backend middleware.

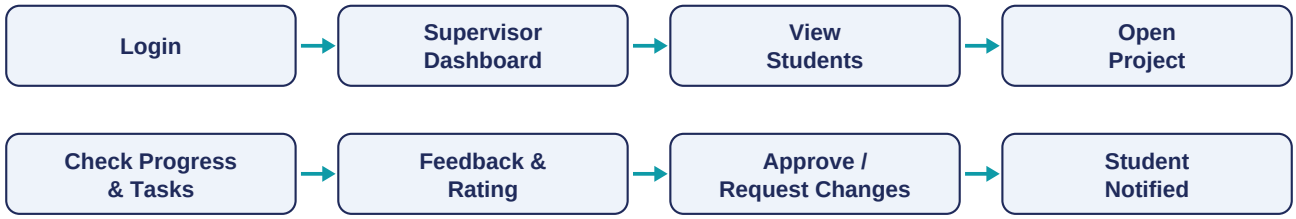
Role	Main Purpose	Key Capabilities
Student	Builds projects	Register and complete profile; create projects; add technologies and team members; create and manage tasks; update progress; submit for review; join teams; view feedback, notifications and leaderboard position.
Supervisor	Reviews projects	View assigned students and their projects; check progress and tasks; give feedback and rating; approve or request changes; track pending reviews.
Admin	Manages the platform	View platform statistics and analytics; manage users (view, search, suspend, activate); moderate, approve, feature or remove projects; handle reports; manage categories, technologies and leaderboard.

6. System Workflows

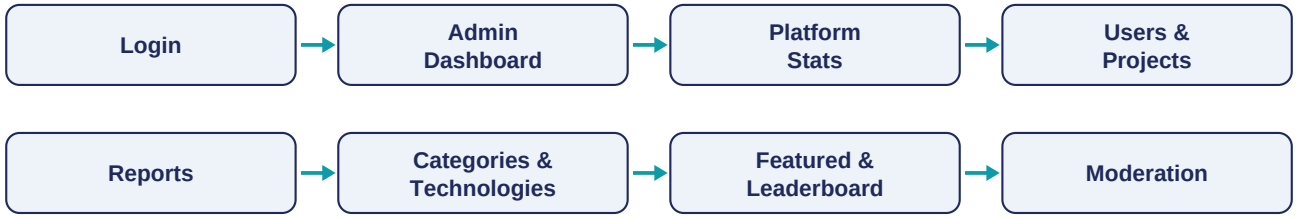
6.1 Student Workflow



6.2 Supervisor Workflow



6.3 Admin Workflow



After a supervisor reviews a project, it is either **approved** and published to the Explore page and rankings, or marked **changes required**, in which case the student updates it and resubmits.

7. Key Features

Module	Features
Authentication	Registration, login, JWT tokens, bcrypt password hashing, role-based redirect, protected routes.
Project Management	Create, edit, delete projects with category, technologies, team, GitHub and live links, logo, screenshots, status and dates.
Task Management	Tasks with priority, status (todo / in-progress / completed), assignee and due date. Progress = Completed Tasks / Total Tasks x 100.
Explore and Community	Search, category, technology and status filters, sorting, pagination, likes, view counts, comments, bookmarks, developer profiles.
Supervisor Review	Technical, UI, functionality and documentation scores, overall score, written feedback, approve or request changes.
Admin Panel	User management, project moderation, reports, featured projects, categories and technologies.
Leaderboards	Top projects and top developers, calculated on the backend.
Notifications and Analytics	Alerts for submissions, approvals, feedback, comments, likes and team invitations; Recharts dashboards for each role.

8. System Architecture and Technology Stack

Layer	Technology
Frontend	React.js (Vite), Tailwind CSS, React Router, Axios, Lucide React, Recharts
Backend	Node.js, Express.js, JWT, bcrypt, dotenv, cors, validation and centralised error middleware
Database	MongoDB Atlas with Mongoose
Tools	Git and GitHub, Postman, Figma
Deployment	Vercel (frontend), Render or Railway (backend), MongoDB Atlas (database)

9. Database Design

The design keeps the number of collections small and manageable. The main collections are **users, projects, tasks, reviews, comments, notifications, bookmarks, reports, categories** and **technologies**. Documents are linked through MongoDB references (for example, a project references its owner, team members and supervisor).

Collection	Main Fields
User	name, email, password (hashed), role (student / supervisor / admin), profileImage, bio, skills[], github, linkedin, projects[], createdAt
Project	title, description, owner, teamMembers[], category, technologies[], githubUrl, liveUrl, image, screenshots[], status, progress, startDate, endDate, views, likes[], rating, supervisor, reviewStatus
Task	project, title, description, assignedTo, priority, status, dueDate, createdAt
Review	project, supervisor, technicalScore, uiScore, functionalityScore, documentationScore, overallScore, feedback, status
Notification	user, title, message, type, isRead, createdAt

10. Project Score and Security

Project Score is calculated only on the backend, and users can never submit their own score:

Component	Supervisor Rating	Community Rating	Completion	Likes	Views
Weight	30%	20%	20%	15%	15%

Security and role-based access. Passwords are hashed with bcrypt and sessions use JWT. Every protected API passes through *authenticateUser()* and then *checkRole()* before reaching the controller, so access is never controlled by hiding buttons in React alone. Student, supervisor and admin routes (*/student/dashboard*, */supervisor/dashboard*, */admin/dashboard*) are restricted to their own roles.

11. Development Plan (10 Weeks)

Week	Focus	Deliverable
1	Requirements and system design	SRS, wireframes, ER diagram, architecture, API plan, GitHub repository
2	Frontend foundation	Responsive UI skeleton with all major pages and dashboards
3	Backend and authentication	Register, login, JWT, protected and role-based routes
4	Student profile and project CRUD	First complete vertical slice (register to project view)
5	Project details and tasks	Task management and automatic progress calculation
6	Explore and community	Search, filters, likes, views, comments, bookmarks, developer profiles
7	Supervisor system	Complete student-supervisor review workflow
8	Admin and leaderboard	User and project management, reports, rankings
9	Notifications, analytics, polish	Feature-complete application with polished UI
10	Testing, deployment, documentation	Tested, deployed app with README, SRS, API docs, PPT and demo

12. Testing and Deployment

Testing covers functional testing of every module, authorization testing (for example, a student calling an admin API must be rejected, while an admin calling an admin API must succeed), UI testing on mobile, tablet and desktop and in Chrome and Edge, and backend cleanup of validation, error handling and environment variables. The frontend is deployed on Vercel, the backend on Render or Railway, and the database on MongoDB Atlas.

13. Expected Outcomes

- A working, deployed MERN application supporting students, supervisors and admins.
- Structured, transparent project review with recorded feedback and scores.
- A showcase of approved projects, rankings and developer profiles available to all registered users.
- Practical experience in JWT authentication, role-based authorization, REST API design, MongoDB references, scoring logic and deployment.

14. Future Scope

- GitHub integration showing stars, forks, languages and commits.
- AI project assistant that suggests tech stack and modules for an idea.
- Auto-generated project completion certificates after supervisor approval.
- Public developer portfolio pages generated from profile data.
- Real-time chat, mobile app and recommendation engine in later versions.

15. Conclusion

ProjectHub brings project creation, task tracking, supervisor review, administration and community showcase into one organised platform. By limiting the scope to three well-defined roles and a manageable set of features, the project can be completed in 10 weeks while still demonstrating authentication, authorization, database design, analytics and deployment, making it a strong full-stack portfolio project.